信 息 工 程 学 院

《深度学习》实验报告

学院：信息工程学院　**班级**：数科 2102　**姓名**：陈　**学号**:202101　**成绩**:

一、实验题目

循环神经网络

二、实验目的

了解 LSTM 模型

了解 RNN 网络

了解 Language Model

三、实验平台

希冀平台

Jupyter

四、实验过程及步骤

导入相关模块并查看版本

实验报告的内容与格式按任课教师的要求书写。

1.import 相关模块并查看版本

In [1]:
```
import matplotlib as mpl
import matplotlib.pyplot as plt
#%matplotlib inline
import numpy as np
import sklearn
import pandas as pd

import os
import sys
import time
import tensorflow as tf
from tensorflow import keras

print(tf.__version__)
print(sys.version_info)

for module in mpl,np,pd,sklearn,tf,keras:
    print(module.__name__,module.__version__)

gpus = tf.config.experimental.list_physical_devices('GPU')
print(gpus)
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)
```

```
2.2.0
sys.version_info(major=3, minor=6, micro=8, releaselevel='final', serial=0)
matplotlib 3.3.4
numpy 1.18.5
pandas 1.0.4
sklearn 0.23.1
tensorflow 2.2.0
tensorflow.keras 2.3.0-tf
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU'), Physi
calDevice(name='/physical_device:GPU:2', device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:3', device_type='GPU')]
```

读取数据集

2.读取数据集

通过执行代码我们看到数据的长度以及前十个数据:

In [2]:
```
input_filepath = 'lstm_sim/shakespeare.txt'
text = open(input_filepath,'r').read()
print(len(text))
print(text[0:10])
```

```
1115394
First Citi
```

数据处理

1.生成词汇表 可以看到数据集中有65个字符。

In [3]:
```
vocab = sorted(set(text)) # 利用set方法取出字符，并将重复字符去掉，sort方法用来排序
print(len(vocab))
print(vocab)
```

```
65
['\n', ' ', '!', '$', '&', "'", ',', '-', '.', '3', ':', ';', '?', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',
'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
```

2. 建立映射

In [4]:
```
# 建立映射 字符-->id
char2idx = {char : idx for idx,char in enumerate(vocab)}
print(char2idx)
```

```
{'\n': 0, ' ': 1, '!': 2, '$': 3, '&': 4, "'": 5, ',': 6, '-': 7, '.': 8, '3': 9, ':': 10, ';': 11, '?': 12, 'A': 13, 'B': 14, 'C':
15, 'D': 16, 'E': 17, 'F': 18, 'G': 19, 'H': 20, 'I': 21, 'J': 22, 'K': 23, 'L': 24, 'M': 25, 'N': 26, 'O': 27, 'P': 28, 'Q': 29,
'R': 30, 'S': 31, 'T': 32, 'U': 33, 'V': 34, 'W': 35, 'X': 36, 'Y': 37, 'Z': 38, 'a': 39, 'b': 40, 'c': 41, 'd': 42, 'e': 43, 'f':
44, 'g': 45, 'h': 46, 'i': 47, 'j': 48, 'k': 49, 'l': 50, 'm': 51, 'n': 52, 'o': 53, 'p': 54, 'q': 55, 'r': 56, 's': 57, 't': 58,
'u': 59, 'v': 60, 'w': 61, 'x': 62, 'y': 63, 'z': 64}
```

In [5]:
```
# 把vocab转为numpy 即id ——> 字符
idx2char = np.array(vocab)
print(idx2char)
```

```
['\n' ' ' '!' '$' '&' "'" ',' '-' '.' '3' ':' ';' '?' 'A' 'B' 'C' 'D' 'E'
 'F' 'G' 'H' 'I' 'J' 'K' 'L' 'M' 'N' 'O' 'P' 'Q' 'R' 'S' 'T' 'U' 'V' 'W'
 'X' 'Y' 'Z' 'a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j' 'k' 'l' 'm' 'n' 'o'
 'p' 'q' 'r' 's' 't' 'u' 'v' 'w' 'x' 'y' 'z']
```

接下来我们要根据映射将text变为对应的idx组成的文本，代码如下：

通过执行代码我们可以看到text_as_int的前十行：

In [6]:
```
# 将text变为对应的idx组成的文本
text_as_int = np.array([char2idx[c] for c in text])
print(text_as_int[0:10])
print(text[0:10])
```

```
[18 47 56 57 58  1 15 47 58 47]
First Citi
```

```
First Citi
```

3. 定义输入与输出 接下来我们定义一个函数来实现输入与输出：

In [7]:
```
# 定义输入与输出
def split_input_target(id_text):
    """
    文本为abcde,则输入为abcd,四个字符对应的输出分别为：bcde,即每个输出都是输入的下一个字符
    """
    return id_text[0:-1],id_text[1:]
```

定义 dataset

这里要将数据转为dataset以方便对数据进行预处理与训练。

1、定义dataset

这里的seq_dataset是将字符数据集char_dataset以seq_length+1的长度批量处理的句子数据集，之所以长度为seq_length+1是因为为了与之前的建立输入输出的函数连上，以便输入与输出的长度都为seq_length。

代码执行结果如下：

In [8]:
```
# 定义dataset
# 每个字符集对应的idx的dataset
char_dataset = tf.data.Dataset.from_tensor_slices(text_as_int)

# 定义一个句子型的dataset
seq_length = 100
seq_dataset = char_dataset.batch(seq_length+1, drop_remainder=True)

# 打印两个数据集的内容
for ch_id in char_dataset.take(2):
    print(ch_id,idx2char[ch_id.numpy()])

for seg_id in seq_dataset.take(2):
    print(seg_id)
    print(repr(''.join(idx2char[seg_id.numpy()])))
```

```
tf.Tensor(18, shape=(), dtype=int64) F
tf.Tensor(47, shape=(), dtype=int64) i
tf.Tensor(
[18 47 56 57 58  1 15 47 58 47 64 43 52 10  0 14 43 44 53 56 43  1 61 43
  1 54 56 53 41 43 43 42  1 39 52 63  1 44 59 56 58 46 43 56  6  1 46 43
 39 56  1 51 43  1 57 54 43 39 49  8  0  0 13 50 50 10  0 31 54 43 39 49
  6  1 57 54 43 39 49  8  0  0 18 47 56 57 58  1 15 47 58 47 64 43 52 10
  0 37 53 59  1], shape=(101,), dtype=int64)
'First Citizen:\nBefore we proceed any further, hear me speak.\n\nAll:\nSpeak, speak.\n\nFirst Citizen:\nYou '
tf.Tensor(
[39 56 43  1 39 50 50  1 56 43 57 53 50 60 43 42  1 56 39 58 46 43 56  1
 58 53  1 42 47 43  1 58 46 39 52  1 58 53  1 44 39 51 47 57 46 12  0  0
 13 50 50 10  0 30 43 57 53 50 60 43 42  8  1 56 43 57 53 50 60 43 42  8
  0  0 18 47 56 57 58  1 15 47 58 47 64 43 52 10  0 18 47 56 57 58  6  1
 63 53 59  1 49], shape=(101,), dtype=int64)
'are all resolved rather to die than to famish?\n\nAll:\nResolved. resolved.\n\nFirst Citizen:\nFirst, you k'
```

2、建立输入与输出

将我们之前定义的输入输出函数应用到数据集上。

以前两个句子为例，我们查看下输入与输出。

In [9]:
```
seq_dataset = seq_dataset.map(split_input_target)
for item_input, item_output in seq_dataset.take(2):
    print(item_input.numpy())
    print(item_output.numpy())
```

```
[18 47 56 57 58  1 15 47 58 47 64 43 52 10  0 14 43 44 53 56 43  1 61 43
  1 54 56 53 41 43 43 42  1 39 52 63  1 44 59 56 58 46 43 56  6  1 46 43
 39 56  1 51 43  1 57 54 43 39 49  8  0  0 13 50 50 10  0 31 54 43 39 49
  6  1 57 54 43 39 49  8  0  0 18 47 56 57 58  1 15 47 58 47 64 43 52 10
  0 37 53 59]
[47 56 57 58  1 15 47 58 47 64 43 52 10  0 14 43 44 53 56 43  1 61 43  1
 54 56 53 41 43 43 42  1 39 52 63  1 44 59 56 58 46 43 56  6  1 46 43 39
 56  1 51 43  1 57 54 43 39 49  8  0  0 13 50 50 10  0 31 54 43 39 49  6
  1 57 54 43 39 49  8  0  0 18 47 56 57 58  1 15 47 58 47 64 43 52 10  0
 37 53 59  1]
[39 56 43  1 39 50 50  1 56 43 57 53 50 60 43 42  1 56 39 58 46 43 56  1
 58 53  1 42 47 43  1 58 46 39 52  1 58 53  1 44 39 51 47 57 46 12  0  0
 13 50 50 10  0 30 43 57 53 50 60 43 42  8  1 56 43 57 53 50 60 43 42  8
  0  0 18 47 56 57 58  1 15 47 58 47 64 43 52 10  0 18 47 56 57 58  6  1
 63 53 59  1]
[56 43  1 39 50 50  1 56 43 57 53 50 60 43 42  1 56 39 58 46 43 56  1 58
 53  1 42 47 43  1 58 46 39 52  1 58 53  1 44 39 51 47 57 46 12  0  0 13
 50 50 10  0 30 43 57 53 50 60 43 42  8  1 56 43 57 53 50 60 43 42  8  0
  0 18 47 56 57 58  1 15 47 58 47 64 43 52 10  0 18 47 56 57 58  6  1 63
 53 59  1 49]
```

3、混排与batch_size

In [10]:
```
batch_size = 64
buffer_size = 10000
seq_dataset = seq_dataset.shuffle(buffer_size).batch(batch_size,drop_remainder=True)
```

定义模型

5.定义模型

这里的Embedding层是用来将每个字符编码为长度为256的向量，因为每个句子是不定长的，所以我们的模型输入大小为[batch_size,None],其中None为句子的长度。

LSTM层中的units为经LSTM之后每个字符将会转化为长度为rnn_units的向量。return_sequence=True意味着我们的LSTM层要输出的依然为一个序列值。

代码执行结果如下：

In [11]:
```
# 定义模型
vocab_size = len(vocab)
embedding_dim = 256
rnn_units = 1024
def build_model(vocab_size, embedding_dim, rnn_units, batch_size):
    model = keras.models.Sequential([
        keras.layers.Embedding(vocab_size, embedding_dim,
                               batch_input_shape=[batch_size, None]),
        keras.layers.LSTM(units=rnn_units,
                          stateful=True,
                          recurrent_initializer="glorot_uniform",
                          return_sequences=True),
        keras.layers.Dense(vocab_size)
    ])
    return model
model =  build_model(vocab_size=vocab_size,
                     embedding_dim=embedding_dim,
                     rnn_units=rnn_units,
                     batch_size=batch_size)
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (64, None, 256)           16640
_________________________________________________________________
lstm (LSTM)                  (64, None, 1024)          5246976
_________________________________________________________________
dense (Dense)                (64, None, 65)            66625
=================================================================
Total params: 5,330,241
Trainable params: 5,330,241
Non-trainable params: 0
_________________________________________________________________
```

在训练之前我们先通过这个未被训练过的模型来 查看下我们的输出是什么样子。

代码如下：

可以看到输出的句子长度为100，64为batch_size，65为每个vocab对应的概率分布。

In [12]:
```
# 查看输出
for input_example_batch, target_example_batch in seq_dataset.take(1):
    example_batch_prediction = model(input_example_batch)
    print(example_batch_prediction.shape)
```

```
(64, 100, 65)
```

然后我们需要根据每个vocab的概率分布进行随机采样。代码如下:

tf.random.categorical的作用即是用来随机采样的。将example_batch_prediction的第一个数据输入进去，num_sample = 1指的是只在65个概率分布中取一个值作为输出。

因输入大小为（100,65），随机 采样后的结果大小为（100,1）。然后我们再通过tf.squeeze来将输出的第二个维度去掉，即最终输出结果大小为（100，）。

In [13]:
```
# random sampling
sample_indices = tf.random.categorical(logits = example_batch_prediction[0],
                       num_samples=1)
print(sample_indices)
sample_indices = tf.squeeze(sample_indices, axis=-1)
print(sample_indices)
```

```
tf.Tensor(
[[43]
 [24]
 [52]
 [11]
 [ 2]
 [39]
 [33]
 [24]
 [52]
 [15]
 [42]
 [22]
 [54]
 [21]
 [17]
 [56]
 [43]
 [ 1]
```

最后我们再打印下这些idx所对应的字符，代码如下：

可以看到虽然我们的预测结果很差（因为还没训练），但是总体逻辑已经行得通了。

In [14]:
```
# 打印输入，输出与预测对应的字符串
print("Input:",repr("".join(idx2char[input_example_batch[0]])))
print()
print("Output:",repr("".join(idx2char[target_example_batch[0]])))
print()
print("Predictions:",repr("".join(idx2char[sample_indices])))
```

```
Input: "y, he is so made on here within, as if he were son\nand heir to Mars; set at upper end o' the table; "

Output: ", he is so made on here within, as if he were son\nand heir to Mars; set at upper end o' the table; n"

Predictions: "eLn;!aULnCdJpIEre $;\nPOD:hjup.SeOoR-j;?IUNd3a;IXm'\nP;JkNht,WN.aebG3;npFW\nHfvkbJAVY&J.!eze3bFs?hz xRn"
```

训练

1、自定义损失函数 在这里我们先自定义一个损失函数：

通过执行代码我们可以看到之前例子的损失函数大小：

In [15]:
```
# 自定义损失函数
def loss(labels,logits):
    return keras.losses.sparse_categorical_crossentropy(
            labels,logits,from_logits=True)
model.compile(optimizer='adam',loss=loss)

# 查看之前例子的loss
example_loss = loss(target_example_batch,example_batch_prediction)
print(example_loss.shape)
print(example_loss.numpy().mean())
```

```
(64, 100)
4.1739626
```

2、模型保存 我们定义一个callback函数以保存训练模型。

In [16]:
```
# 模型保存与训练
output_dir = "lstm_sim/text_generation_lstm_checkpoints"
if not os.path.exists(output_dir):
    os.mkdir(output_dir)
checkpoint_prefix = os.path.join(output_dir,'ckpt_{epoch}')
checkpoint_callback = keras.callbacks.ModelCheckpoint(
                filepath=checkpoint_prefix,
                save_weights_only=True)
```

3、训练

In [17]:
```
epochs = 10
history = model.fit(seq_dataset,epochs = epochs,
                    callbacks=[checkpoint_callback])
```

```
Epoch 1/10
172/172 [==============================] - 11s 63ms/step - loss: 2.5557
Epoch 2/10
172/172 [==============================] - 11s 62ms/step - loss: 1.8606
Epoch 3/10
172/172 [==============================] - 11s 62ms/step - loss: 1.6173
Epoch 4/10
172/172 [==============================] - 11s 63ms/step - loss: 1.4894
Epoch 5/10
172/172 [==============================] - 11s 63ms/step - loss: 1.4113
Epoch 6/10
172/172 [==============================] - 11s 64ms/step - loss: 1.3564
Epoch 7/10
172/172 [==============================] - 11s 64ms/step - loss: 1.3109
Epoch 8/10
172/172 [==============================] - 11s 64ms/step - loss: 1.2721
Epoch 9/10
172/172 [==============================] - 11s 65ms/step - loss: 1.2345
Epoch 10/10
172/172 [==============================] - 11s 65ms/step - loss: 1.1986
```

训练完毕后保存最新的模型：

In [18]:
```
tf.train.latest_checkpoint(output_dir)
```

Out[18]:
```
'lstm_sim/text_generation_lstm_checkpoints/ckpt_10'
```

文本生成

7.文本生成

我们首先再定义一个模型来使用最近保存的模型权重值。

In [19]:
```
model2 = build_model(vocab_size,
                     embedding_dim,
                     rnn_units,
                     batch_size=1)

model2.load_weights(tf.train.latest_checkpoint(output_dir))

# 定义model2的输入shape
model2.build(tf.TensorShape([1,None]))
model2.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_1 (Embedding)      (1, None, 256)            16640
_________________________________________________________________
lstm_1 (LSTM)                (1, None, 1024)           5246976
_________________________________________________________________
dense_1 (Dense)              (1, None, 65)             66625
=================================================================
Total params: 5,330,241
Trainable params: 5,330,241
Non-trainable params: 0
_________________________________________________________________
```

最后我们定义一个函数来实现文本生成。

通过执行代码，我们可以得到当输入为"first: "时的输出结果为：

In [20]:
```
# 定义函数实现文本生成
temperature = 0.5

def generate_text(model,start_string,num_generate=1000):
    input_eval = [char2idx[ch] for ch in start_string]
    input_eval = tf.expand_dims(input_eval,0)

    text_generated=[]
    model.reset_states()

    for _ in range(num_generate):
        predictions = model(input_eval)
        predictions = predictions / temperature
        predictions = tf.squeeze(predictions,0)
        predicted_id = tf.random.categorical(
            predictions,num_samples=1)[-1,0].numpy()
        text_generated.append(idx2char[predicted_id])
        input_eval = tf.expand_dims([predicted_id],0)
    return start_string+''.join(text_generated)

# 调用
new_text = generate_text(model2,"first: ")
print(new_text)
```

```
first: to find you
As if they say the wars of the people,
And not our good dark conduct his means to be a brave strength and noble eyes of heavy tongue,
With whom he was more than the face of the
hour of the instrument,
That lies upon the seat of the air of the shade,
And cry 'Courage, speak to the people,
Who is come to make his son's are not of the market-place.

HORTENSIO:
Why, ha! it is the soldiers of thy face?

KING EDWARD IV:
Welcome, good night: but he shall swear to them?

BRAKENBURY:
I do believe thee and call him here,
And now I take my leave to do them strength
To the sentence of the time to her his foul action.
```

可以看出生成的文本虽然含义有些问题，但整体还是较为连贯的

RNN 开源项目

```python
import numpy as np
import torch
from torch import device
import torch.nn as nn
# from train import char2int, one_hot_encode, dict_size, int2char, model


class Model(nn.Module):
    """
    input_size (int):输入数据的特征大小，即每个时间步的输入向量的维度。
    hidden_size (int):隐藏层的特征大小，即每个时间步的隐藏状态向量的维度。
    num_layers (int,可选):RNN的层数，默认值为1。当层数大于1时，RNN会变为多层RNN。
    nonlinearity (str,可选):指定激活函数，默认值为'tanh'。可选值有'tanh'和'relu'。
    bias (bool,可选):如果设置为True，则在RNN中添加偏置项。默认值为True。
    batch_first (bool,可选):如果设置为True，则输入数据的形状为(batch_size, seq_len, input_size)。否则，默认输入数据的形状为(seq_len, batch_si
    dropout (float,可选):如果非零，则在除最后一层之外的每个RNN层之间添加dropout层，其丢弃概率为dropout。默认值为0。
    bidirectional (bool,可选):如果设置为True，则使用双向RNN。默认值为False。
    """
    def __init__(self, input_size, output_size, hidden_dim, n_layers):
        super(Model, self).__init__()

        # Defining some parameters
        self.hidden_dim = hidden_dim # 隐藏状态 ht 的维度
        self.n_layers = n_layers # 网络的层数

        # Defining the layers
        # RNN Layer
        self.rnn = nn.RNN(input_size, hidden_dim, n_layers, batch_first=True)
        # Fully connected layer
        self.fc = nn.Linear(hidden_dim, output_size)

    def forward(self, x):
        batch_size = x.size(0)

        # Initializing hidden state for first input using method defined below
        hidden = self.init_hidden(batch_size)

        # Passing in the input and hidden state into the model and obtaining outputs
        out, hidden = self.rnn(x, hidden)

        # Reshaping the outputs such that it can be fit into the fully connected layer
        out = out.contiguous().view(-1, self.hidden_dim)
        out = self.fc(out)

        return out, hidden

    def init_hidden(self, batch_size):
        # This method generates the first hidden state of zeros which we'll use in the forward pass
        # We'll send the tensor holding the hidden state to the device we specified earlier as well
        hidden = torch.zeros(self.n_layers, batch_size, self.hidden_dim)
        return hidden
```

```python
text = ['hey how are you', 'good i am fine', 'have a nice day']
chars = set(''.join(text))
int2char = dict(enumerate(chars))
char2int = {char: ind for ind, char in int2char.items()}
dict_size = len(char2int)
model = Model(input_size=dict_size, output_size=dict_size, hidden_dim=12, n_layers=1)

def one_hot_encode(sequence, dict_size, seq_len, batch_size):
    # Creating a multi-dimensional array of zeros with the desired output shape
    features = np.zeros((batch_size, seq_len, dict_size), dtype=np.float32)

    # Replacing the 0 at the relevant character index with a 1 to represent that character
    for i in range(batch_size):
        for u in range(seq_len):
            features[i, u, sequence[i][u]] = 1
    return features


def predict(model, character):
    # One-hot encoding our input to fit into the model
    character = np.array([[char2int[c] for c in character]])
    character = one_hot_encode(character, dict_size, character.shape[1], 1)
    character = torch.from_numpy(character)
    # character.to(device)

    out, hidden = model(character)

    prob = nn.functional.softmax(out[-1], dim=0).data

    # Taking the class with the highest probability score from the output
    char_ind = torch.max(prob, dim=0)[1].item()

    return int2char[char_ind], hidden

# This function takes the desired output length and input characters as arguments, returning the produced sentence
def sample(model, out_len, start='hey'):
    model.eval() # eval mode
    start = start.lower()
    # First off, run through the starting characters
    chars = [ch for ch in start]
    size = out_len - len(chars)
    # Now pass in the previous characters and get a new one
    for ii in range(size):
        char, h = predict(model, chars)
        chars.append(char)

    return ''.join(chars)
```

```python
text = ['hey how are you', 'good i am fine', 'have a nice day']

# Join all the sentences together and extract the unique characters from the combined sentences
# 将所有句子连接在一起，并从组合的句子中提取独特的字符。
chars = set(''.join(text))

int2char = dict(enumerate(chars))
# print(int2char)

# Creating another dictionary that maps characters to integers
char2int = {char: ind for ind, char in int2char.items()}

maxlen = len(max(text, key=len))

for i in range(len(text)):
  while len(text[i])<maxlen:
      text[i] += ' '

# 由于我们要在每个时间步预测序列中的下一个字符，我们必须将每个句子分为：

# 输入数据
# 最后一个字符需排除因为它不需要作为模型的输入
# 目标/真实标签
# 它为每一个时刻后的值，因为这才是下一个时刻的值。
# Creating lists that will hold our input and target sequences
input_seq = []
target_seq = []

for i in range(len(text)):
    # Remove last character for input sequence
    input_seq.append(text[i][:-1])

    # Remove first character for target sequence
    target_seq.append(text[i][1:])
    print("Input Sequence: {}\nTarget Sequence: {}".format(input_seq[i], target_seq[i]))

# 现在我们可以通过使用上面创建的字典映射输入和目标序列到整数序列。
# 这将允许我们随后对输入序列进行一次one-hot encoding。

for i in range(len(text)):
    input_seq[i] = [char2int[character] for character in input_seq[i]]
    target_seq[i] = [char2int[character] for character in target_seq[i]]

# 定义如下三个变量
# dict_size: 字典的长度，即唯一字符的个数。它将决定one-hot vector的长度
# seq_len:输入到模型中的sequence长度。这里是最长的句子的长度-1，因为不需要最后一个字符
# batch_size: mini batch的大小，用于批量训练
dict_size = len(char2int)
seq_len = maxlen - 1
batch_size = len(text)
```

```python
def one_hot_encode(sequence, dict_size, seq_len, batch_size):
    # Creating a multi-dimensional array of zeros with the desired output shape
    features = np.zeros((batch_size, seq_len, dict_size), dtype=np.float32)

    # Replacing the 0 at the relevant character index with a 1 to represent that character
    for i in range(batch_size):
        for u in range(seq_len):
            features[i, u, sequence[i][u]] = 1
    return features
# 同时定义一个helper function，用于初始化one-hot向量
# Input shape --> (Batch Size, Sequence Length, One-Hot Encoding Size)
input_seq = one_hot_encode(input_seq, dict_size, seq_len, batch_size)

# 到此我们完成了所有的数据预处理，可以将数据从NumPy数组转为PyTorch张量啦
input_seq = torch.from_numpy(input_seq)
target_seq = torch.Tensor(target_seq)

# 接下来就是搭建模型的步骤，你可以在这一步使用全连接层，卷积层，RNN层，LSTM层等等。
# 但是我在这里使用最最基础的nn.rnn来示例一个RNN是如何使用的。
# from RNN_model import Model

# 要开始构建我们自己的神经网络模型，我们可以为所有神经网络模块定义一个继承 PyTorch 的基类（nn.module）的类。
# 这样做之后，我们可以开始在构造函数下定义一些变量以及模型的层。 对于这个模型，我们将只使用一层 RNN，然后是一个全连接层。 全连接层将负责将 RNN
# 我们还必须将 forward() 下的前向传递函数定义为类方法。 前向函数是按顺序执行的，因此我们必须先将输入和零初始化隐藏状态通过 RNN 层，然后再将 RNN
# 我们必须定义的最后一个方法是我们之前调用的用于初始化hidden state的方法 - init_hidden()。 这基本上会在我们的隐藏状态的形状中创建一个零张量。



# 在定义了上面的模型之后，我们必须用相关参数实例化模型并定义我们的超参数。 我们在下面定义的超参数是：
# n_epochs: 模型训练所有数据集的次数
# lr: learning rate学习率

# 与其他神经网络类似，我们也必须定义优化器和损失函数。 我们将使用 CrossEntropyLoss，因为最终输出基本上是一个分类任务和常见的 Adam 优化器。
# Instantiate the model with hyperparameters
model = Model(input_size=dict_size, output_size=dict_size, hidden_dim=12, n_layers=1)
# We'll also set the model to the device that we defined earlier (default is CPU)
# model.to(device)
# Define hyperparameters
n_epochs = 100 # 训练轮数
lr = 0.01 # 学习率
# Define Loss, Optimizer
loss_fn = nn.CrossEntropyLoss() # 交叉熵损失函数
optimizer = torch.optim.Adam(model.parameters(), lr=lr) # 采用Adam作为优化器
# 现在我们可以开始训练了！
# 由于我们只有几句话，所以这个训练过程非常快。
# 然而，随着我们的进步，更大的数据集和更深的模型意味着输入数据要大得多，并且我们必须计算的模型中的参数数量要多得多。
# Training Run
for epoch in range(1, n_epochs + 1):
    optimizer.zero_grad()  # Clears existing gradients from previous epoch
    # input_seq.to(device) # 使用GPU
    output, hidden = model(input_seq)
```

```
Input Sequence: hey how are yo
Target Sequence: ey how are you
Input Sequence: good i am fine
Target Sequence: ood i am fine
Input Sequence: have a nice da
Target Sequence: ave a nice day
Epoch: 10/100............. Loss: 2.3903
Epoch: 20/100............. Loss: 2.1294
Epoch: 30/100............. Loss: 1.7537
Epoch: 40/100............. Loss: 1.3361
Epoch: 50/100............. Loss: 0.9693
Epoch: 60/100............. Loss: 0.6931
Epoch: 70/100............. Loss: 0.4963
Epoch: 80/100............. Loss: 0.3590
Epoch: 90/100............. Loss: 0.2642
Epoch: 100/100............. Loss: 0.2010
```

```
In [4]: print(sample(model, 15, 'good'))
        print(sample(model, 15, 'h'))
        print(sample(model, 15, 'you'))
```

```
good i am fine
hey how are you
youday fine fic
```

可以发现，使用 text = ['hey how are you', 'good i am fine', 'have a nice day']作为训练数据后，在输入 good、h 的时候都能成功预测，但在输入 you 时则不行，主要问题在于训练样本小，出现了过拟合的现象；且输入的 you 没有出现在词汇表中

五、实验总结

通过本次实验，我初步了解了 LSTM 模型，并根据成功运行了一个模型，根据给出的文本生成对应文本，虽然含义可能不是很清晰，但整体上还是较为连贯的。此外，我还部署了一个简单的 RNN 项目，因训练样本较小，其在预测出现过的字符时效果较好，但在预测未出现过的字符时效果较差